

API de Logs — Documento do Sistema

22 de setembro de 2026 · Paulo

Visão geral

A API de Logs recebe, por um único endpoint (`POST /logs`), os logs enviados pelas aplicações de qualquer cliente e os guarda de forma isolada por cliente. É o projeto de testes da equipe na disciplina AI Assisted Development, do MBA em Engenharia de Software e IA.

Cada log informa uma mensagem, um nível (informação, alerta, erro...), um `correlationId` opcional para ligar os logs de um mesmo fluxo e um `informationData` opcional com o objeto que a aplicação estava manipulando. O cliente é identificado pela API key enviada no cabeçalho, nunca por um campo do corpo da requisição.

Escopo desta versão: ingestão de logs, cadastro de clientes, aplicações e usuários, retenção automática por cliente, painel de consulta de logs e alertas.

Stack e arquitetura

A API é escrita em Python com FastAPI e grava no MongoDB pelo driver assíncrono oficial.

Camada	Tecnologia	Papel
API	FastAPI	Rotas, injeção de dependências, autenticação por <code>X-API-Key</code> , Swagger
Validação	Pydantic v2	Modelos de entrada, documento e resposta; rejeita campos extras
Acesso a dados	PyMongo Async (<code>AsyncMongoClient</code>)	Driver oficial; o Motor foi descontinuado em favor dele
Banco	MongoDB	Documentos BSON, índices e TTL para retenção
Senhas	pwdlib + Argon2	Hash das senhas dos usuários da plataforma
Testes	pytest, httpx, Testcontainers	Testes de endpoint contra um MongoDB real em Docker

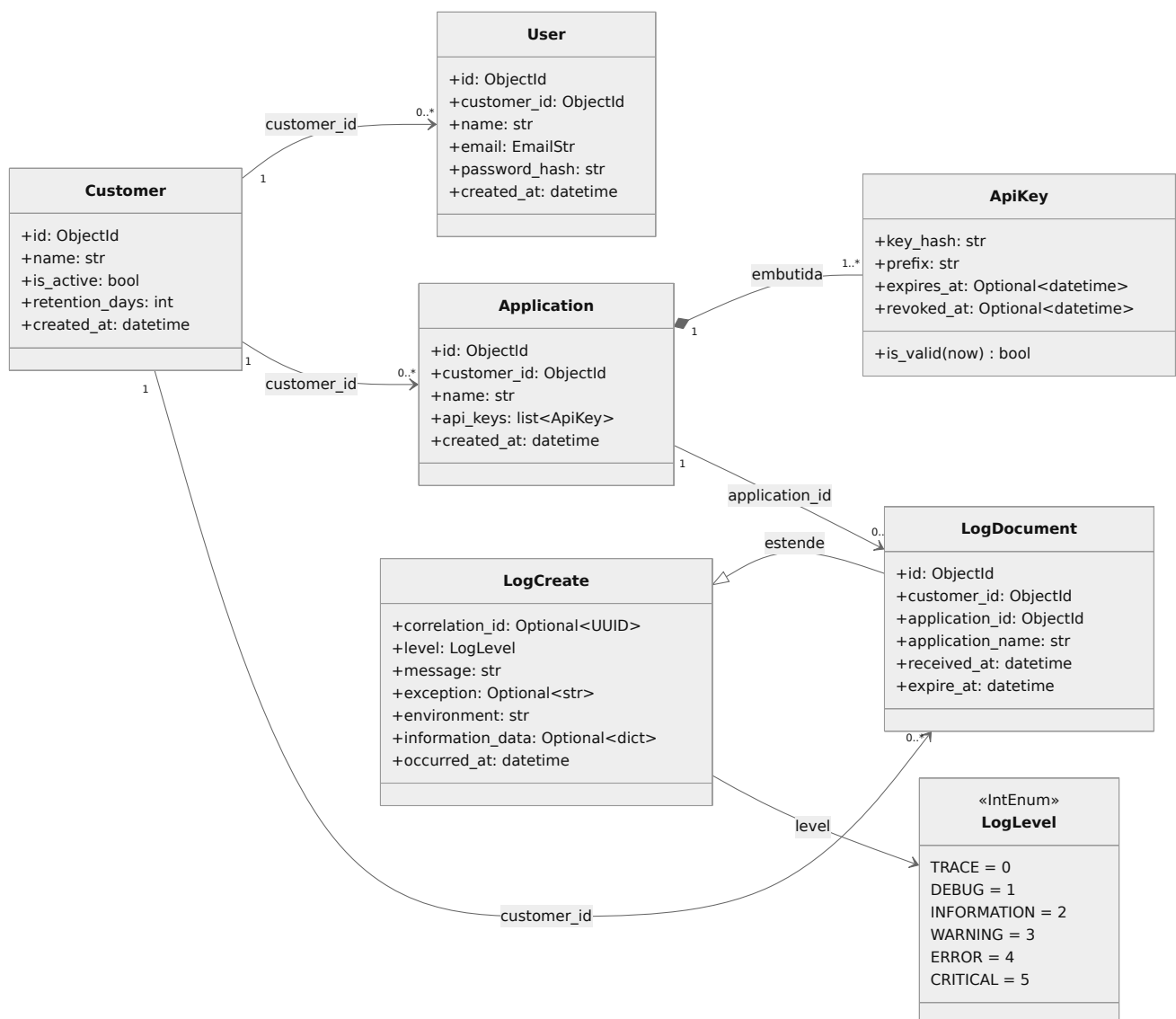
A conexão usa `AsyncMongoClient(uri, uuidRepresentation="standard", tz_aware=True)` , para gravar `UUID` no formato padrão e ler datas sempre com fuso (UTC).

Modelo de domínio

O domínio tem cinco entidades e um enum. Um cliente (`Customer`) tem usuários e aplicações, cada aplicação tem suas API keys e é dona dos logs que envia.

Entidade	O que representa
<code>Customer</code>	Empresa cliente da plataforma; define o prazo de retenção dos seus logs

Entidade	O que representa
User	Pessoa que acessa a plataforma de logs; pertence a um cliente. Não confundir com o usuário final do sistema do cliente
Application	Sistema do cliente que envia logs; substitui o antigo campo livre ApplicationName
ApiKey	Credencial de uma aplicação; guardada só como hash, pode expirar ou ser revogada
LogCreate / LogDocument	O log como o cliente envia e como a API grava
LogLevel	Enum numérico: Trace 0, Debug 1, Information 2, Warning 3, Error 4, Critical 5

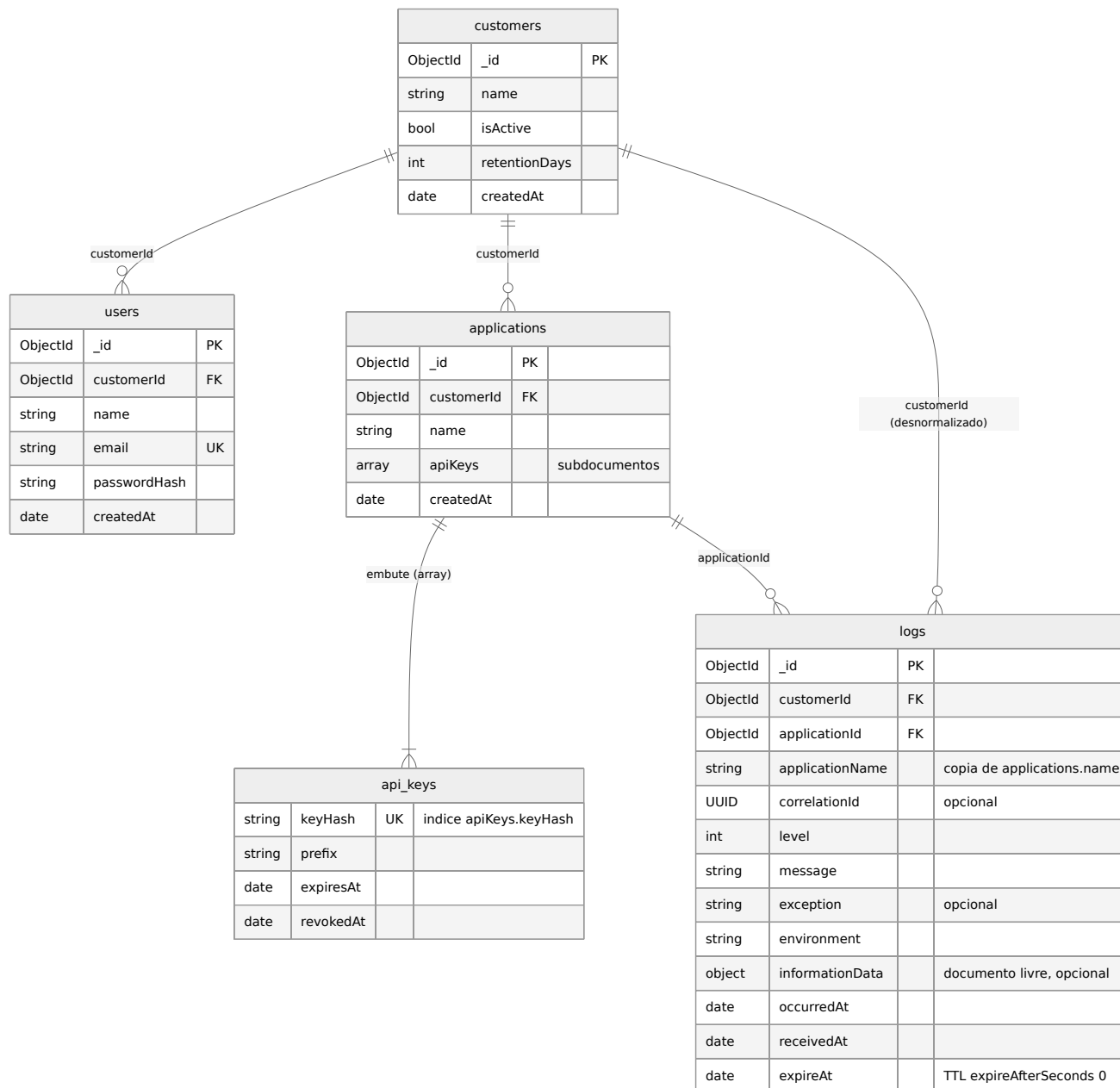


`LogDocument` estende `LogCreate` com os campos que só a API preenche: dono, aplicação, horário de recebimento e data de expiração. Um terceiro modelo, `LogRead`, com `id` como texto, é usado nas respostas.

O log tem duas datas: `occurred_at`, quando o evento aconteceu no cliente, e `received_at`, quando chegou à API. Com o nível numérico, o filtro "a partir de Warning" é um simples `level >= 3`.

Persistência no MongoDB

São quatro coleções: `customers`, `users`, `applications` e `logs`. As API keys ficam embutidas em `applications`, e as demais relações são referências por `objectId`. Como o MongoDB não tem chave estrangeira, a integridade é garantida pela API.



`api_keys` aparece como entidade só para facilitar a leitura. Fisicamente, é o array `apiKeys` dentro de cada documento de `applications`.

Embutir ou referenciar

Relação	Decisão	Motivo
Application → ApiKey	Embutida	Poucas por aplicação e sempre lidas junto com ela
Customer → Application	Referência	A aplicação é consultada sozinha a cada requisição

Relação	Decisão	Motivo
Customer → User	Referência	Se um usuário precisar atender vários clientes, vira um array <code>customerIds</code>
Log → Customer / Application	Referência + cópia do nome	Listagens e filtros não precisam de <code>\$lookup</code>

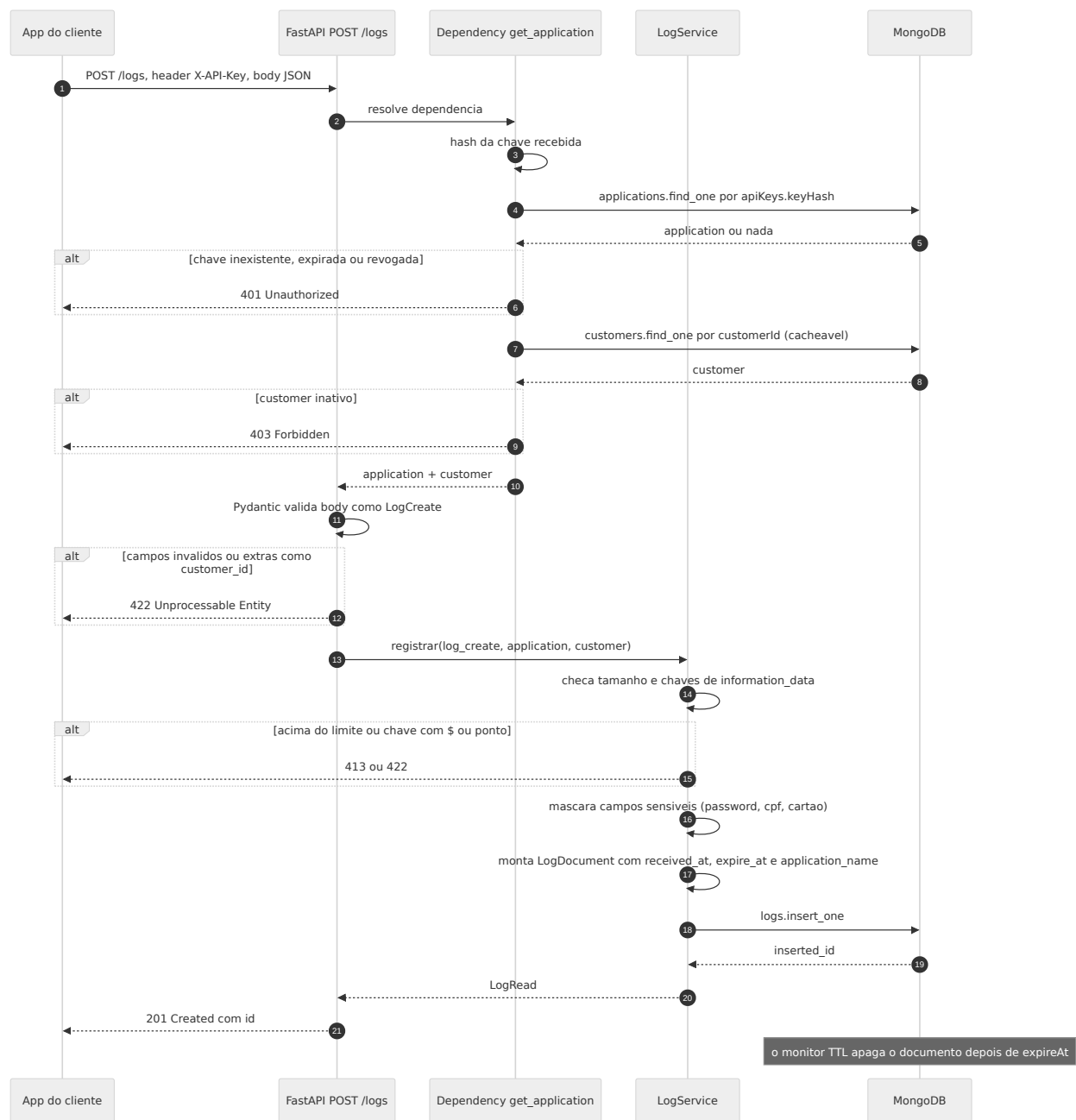
Índices

Coleção	Índice	Uso
<code>users</code>	<code>{ email: 1 }</code> único	Login e unicidade do e-mail
<code>applications</code>	<code>{ "apiKeys.keyHash": 1 }</code> único	Autenticação de cada requisição
<code>logs</code>	<code>{ customerId: 1, occurredAt: -1 }</code>	Listagem padrão, mais recentes primeiro
<code>logs</code>	<code>{ customerId: 1, correlationId: 1 }</code>	Rastrear um fluxo completo
<code>logs</code>	<code>{ customerId: 1, applicationId: 1, level: 1, occurredAt: -1 }</code>	Filtros combinados
<code>logs</code>	<code>{ expireAt: 1 }</code> com <code>expireAfterSeconds: 0</code>	Apagar logs vencidos (TTL)

Retenção. Ao inserir, a API calcula `expireAt = receivedAt + retentionDays` do cliente, e o monitor TTL do MongoDB apaga o documento depois dessa data. Assim cada cliente tem seu próprio prazo. Uma time series collection comprimiria melhor, mas o TTL dela vale para a coleção inteira.

Fluxo de ingestão: POST /logs

Um log passa por autenticação, validação, checagem de tamanho e mascaramento antes de ser gravado. Se tudo der certo, a resposta é `201 Created` com o id do log.



Cada bloco `alt` é uma rejeição com seu próprio código HTTP:

Código	Quando acontece
201 Created	Log gravado; o corpo traz o id
401 Unauthorized	API key ausente, inexistente, expirada ou revogada
403 Forbidden	O cliente dono da chave está inativo
413 Payload Too Large	<code>information_data</code> acima do limite de tamanho
422 Unprocessable Entity	Corpo inválido, campo extra (como <code>customer_id</code>) ou chave com <code>\$</code> ou <code>.</code> em <code>information_data</code>

Segurança e LGPD

O dono de um log é definido só pela API key. Nenhum campo do corpo da requisição consegue trocar o cliente ou a aplicação.

- **API keys:** geradas pela plataforma e mostradas uma única vez. O banco guarda só o hash e um prefixo curto (ex.: `1x_ab12`) para identificação. Cada chave pode expirar ou ser revogada sem afetar as outras da mesma aplicação.
- **Campos extras proibidos:** `LogCreate` usa `ConfigDict(extra="forbid")`, então um `customer_id` no payload é recusado com 422 em vez de ser ignorado.
- **Senhas:** guardadas só como `password_hash` (pwdlib + Argon2), nunca em texto.
- **Isolamento por cliente:** toda consulta de logs filtra por `customerId`, e todos os índices de `logs` começam por esse campo.
- **Dados sensíveis em `information_data`:** como o cliente pode enviar o request inteiro, a API mascara campos como senha, CPF e cartão antes de gravar. O tamanho do objeto tem um limite, e chaves com `$` ou `.` são recusadas.
- **Retenção:** logs não ficam para sempre; o TTL apaga cada documento ao fim do prazo do cliente, o que ajuda no princípio de necessidade da LGPD.
- **Imutabilidade:** logs só são inseridos. Não existe endpoint de alteração.

Estratégia de testes

Os testes de endpoint rodam com pytest e `httpx.AsyncClient` contra um MongoDB real subido pelo Testcontainers. Mocks não validariam os índices únicos nem o TTL.

Caso	Entrada	Resultado esperado
Log válido	Chave válida, corpo completo	201; documento com <code>customerId</code> , <code>applicationName</code> , <code>receivedAt</code> e <code>expireAt</code> preenchidos pela API
Sem API key	Cabeçalho ausente	401
Chave revogada ou expirada	<code>revokedAt</code> ou <code>expiresAt</code> no passado	401
Cliente inativo	Chave válida de cliente com <code>isActive: false</code>	403
Tentativa de trocar o dono	Corpo com <code>customer_id</code>	422 e nada gravado
Nível inválido	<code>level: 9</code>	422
Payload grande	<code>information_data</code> acima do limite	413
Chave proibida	<code>information_data</code> com chave <code>\$where</code> ou <code>a.b</code>	422
Mascaramento	<code>information_data</code> com <code>password</code> e <code>cpf</code>	201; valores mascarados no banco

Caso	Entrada	Resultado esperado
Retenção	Cliente com <code>retentionDays: 7</code>	<code>expireAt</code> = <code>receivedAt</code> + 7 dias
Isolamento	Consulta com usuário do cliente A	Nenhum log do cliente B aparece
E-mail duplicado	Dois usuários com o mesmo e-mail	Segundo cadastro recusado

Decisões e pontos em aberto

O modelo original foi ajustado nestes pontos:

Modelo original	Decisão	Motivo
<code>CustomerId</code> enviado no corpo	Cliente vem da API key	Impede gravar logs em nome de outro cliente
<code>ApplicationName</code> texto livre	Entidade <code>Application</code> + nome copiado no log	Evita "Checkout" e "checkout" como aplicações diferentes
<code>User</code> sem vínculo	<code>User.customerId</code>	Garante que cada usuário veja só os logs do seu cliente
<code>password</code>	<code>password_hash</code> com Argon2	Senha nunca fica em texto
<code>LogId</code> int	<code>ObjectId</code>	Padrão do MongoDB, gerado sem consultar o banco
Log sem data	<code>occurred_at</code> e <code>received_at</code>	Hora do evento e hora do recebimento
LogLevel com 3 valores	6 valores numéricos	Separa erro tratado de falha crítica; filtro por faixa
<code>InformationData</code> object	Documento BSON com limite e mascaramento	Protege dados sensíveis e o tamanho do banco

Em aberto

- ☐ Um usuário pode atender mais de um cliente? Se sim, `customerId` vira `customerIds`.
- ☐ Qual o limite de tamanho de `information_data`? Sugestão: 64 KB.
- ☐ Quais campos entram na lista de mascaramento, e ela é configurável por cliente?
- ☐ A API aceita envio em lote (`POST /logs/batch`) nesta versão?